# Accelerating Attack and Fault Detection in Cyber-Physical Systems Using SmartNICs and DPDK

Samia Choueiri*, Sergio Elizalde*, Ali Mazloum*, Ali AlSabeh‡, Jose Gomez †, Elie Kfoury*, Jorge Crichigno*
*College of Engineering and Computing, University of South Carolina (USC)
‡College of Sciences and Engineering, University of South Carolina Aiken (USCA)
†Katz School of Business, Fort Lewis College, USA.
Email: {choueiri, elizalds, amazloum}@email.sc.edu, ali.alsabeh@usca.edu, jagomez@fortlewis.edu,
ekfoury@email.sc.edu, jcrichigno@cec.sc.edu

***Abstract*—Cyber-physical systems (CPS) require real-time anomaly detection under strict latency constraints, where delayed response can compromise system stability and safety. This paper presents a SmartNIC-accelerated framework for in-network detection of faults and network-based attacks in Modbus/TCP CPS. The proposed system performs feature extraction and Random Forest (RF) inference directly in the data plane using an NVIDIA BlueField-3 SmartNIC and DPDK, enabling low-latency classification without host CPU involvement. Experimental results demonstrate that the proposed approach achieves up to 98% detection accuracy with microsecond-scale inference latency ranging from approximately 0.1 to 66 $\mu$s, while maintaining a low memory footprint (up to 36 MiB) across different model configurations. These results show that accurate and efficient anomaly detection can be performed entirely in the network data plane, enabling scalable, real-time monitoring for latency-critical CPS environments.**

***Index Terms*—Cyber-physical Systems, Cyber-attack Detection, Fault Detection, SmartNICs, Random Forest, DPDK.**

## I. Introduction

Cyber-physical systems (CPS) underpin modern critical infrastructure such as energy, water, manufacturing, and transportation. These systems integrate physical processes with control logic executed on Programmable Logic Controllers (PLCs) and rely on industrial communication networks connecting field devices and Human–Machine Interfaces (HMIs). While Operational Technology (OT) networks were traditionally isolated, increasing IT/OT convergence for remote monitoring and analytics has significantly expanded connectivity and system complexity [1].

Although this convergence improves efficiency, it also introduces new vulnerabilities and failure modes. Network delays, congestion, misconfigurations, and cyber-attacks can disrupt time-sensitive control traffic, leading to instability, actuator misbehavior, or sensor anomalies. Such effects often resemble physical faults, making it difficult to distinguish between faults and malicious activities. Consequently, timely and reliable detection of both faults and attacks is critical for CPS safety and resilience.

Conventional CPS monitoring solutions rely on centralized processing, which increases latency, data movement overhead, and limits scalability in high-speed networks. In contrast, SmartNICs enable programmable in-network processing by offloading computation from the host CPU to the data plane. Prior work has demonstrated their effectiveness in accelerating network monitoring and intrusion detection in latency-sensitive environments [2], making them a promising platform for real-time CPS anomaly detection.

This paper presents a SmartNIC-accelerated anomaly detection framework for Modbus/TCP-based CPS that performs feature extraction and Random Forest (RF) inference directly in the network data plane. A pre-trained RF model is offloaded to an NVIDIA BlueField-3 SmartNIC and executed within a DPDK-enabled datapath, enabling low-latency classification of faults and network-based attacks. The framework leverages lightweight feature extraction and efficient model configurations to achieve high detection accuracy while meeting strict real-time constraints. By executing inference in-network, the system reduces host CPU utilization, minimizes data movement, and enables scalable monitoring at line rate. The source code of this paper is publicly available [3].

The main contributions of this work are as follows:

- A SmartNIC-based framework for real-time anomaly detection in CPS operating directly in the network data plane.
- A lightweight in-network RF model that achieves high detection accuracy with low latency and reduced memory footprint.
- A SHAP-based explainability analysis for interpretable detection in safety-critical CPS environments.
- A comprehensive evaluation on an NVIDIA BlueField-3 SmartNIC, measuring accuracy, inference latency, and memory footprint under realistic traffic conditions.

This work was supported by the U.S. National Science Foundation under awards 2403360 and 2400729.

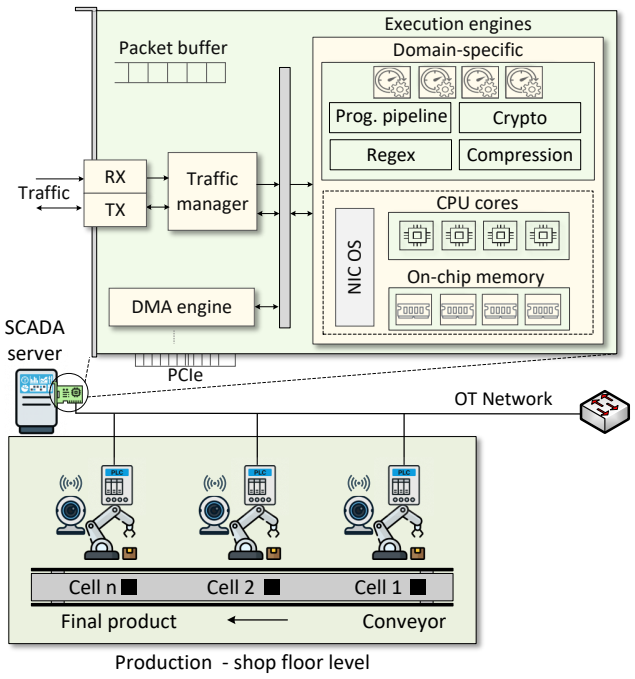


Fig. 1: CPS architecture showing PLC-controlled industrial cells connected via an OT network to a SCADA server equipped with a SmartNIC. The SmartNIC includes programmable data plane components used for high-performance packet processing.

The remainder of this paper is structured as follows. Section II introduces CPS and SmartNIC fundamentals. Section III reviews related work. Section IV presents the system architecture. Section V reports experimental results, and Section VI concludes the paper.

## II. Background

### A. Cyber-Physical Systems (CPSs)

CPSs with Supervisory Control and Data Acquisition (SCADA) represent the most widespread deployments in critical infrastructure sectors such as manufacturing, energy, and water treatment. These systems typically follow a hierarchical architecture that separates supervisory and physical process functions across different layers. At the supervisory layer, the SCADA server provides system monitoring, visualization, and high-level control. The control layer consists of PLCs, which execute real-time control logic and communicate with field devices (robots and machines) using industrial protocols such as Modbus/TCP. The adoption of Industry 4.0 concepts, including real-time data analytics and remote supervision, has increased reliance on IP-based networking within OT environments [4].

As a result, SCADA servers now serve as central aggregation points for high-rate sensor data and control traffic originating from multiple PLCs across the production floor, as shown in Fig. 1. Consequently, there is a growing need for high-performance network components capable of handling protocol processing, traffic steering, and inline analysis without burdening the host CPU. Therefore, the SCADA server with a SmartNIC provides a promising platform to meet these requirements by enabling programmable, data plane acceleration within OT networks.

### B. Overview of SmartNICs and DPDK

SmartNICs are emerging devices that integrate programmable computing resources into the network interface. In general, the architecture of SmartNICs combines general-purpose CPUs with domain-specific processing engines [5]. This architecture may vary from one vendor's design to another. Fig. 1 presents the proposed SmartNIC abstraction used in our CPS architecture, reflecting capabilities commonly available in modern SmartNICs. Network traffic received at the RX NIC port is sent to the traffic manager or NIC switch. The SmartNIC includes execution engines composed of CPU cores for general-purpose tasks and DSAs for high-performance applications. On-chip memory is used to store flow state and intermediate data.

These components work together to offload processing from the host CPU and accelerate networking and security functions. Developers can program the SmartNIC using frameworks such as DPDK to gain direct access to packet queues, flow tables, and eSwitch functions [6]. This capability enables low-latency implementation of security, telemetry, and Machine Learning (ML)-based analytics directly within the network interface.

## III. Related Work

### A. Programmable Data Planes in OT Systems

Recent research has increasingly leveraged programmable data planes to satisfy the stringent latency, determinism, and security requirements of CPS and OT networks. P4-based approaches focus on enforcing security and control logic directly within network switches, enabling line-rate deep packet inspection, stateful validation of legacy industrial protocols such as Modbus/TCP, and ultra-low-latency reaction to safety-critical events [7], [8]. While effective at high throughput, these solutions are constrained by hardware-specific pipelines and limited programmability, restricting their applicability across heterogeneous OT deployments. Complementary work explores eBPF-based packet processing to provide greater flexibility at the network edge, supporting dynamic function chaining, runtime reconfiguration, and complex stateful logic for resilience and asset discovery [9]. However, eBPF executions remain software-based and may struggle to deliver predictable performance under high traffic loads.

Our approach offloads packet processing tasks to a SmartNIC to leverage hardware programmability while employing DPDK as a kernel-bypass mechanism for fast user-space packet processing. This hybrid design aims to merge the deterministic performance of hardware acceleration with the adaptability of software-based processing.

### B. ML for ICS Security and Resilience

Anomaly detection in CPSs is addressed using supervised ML models that classify multi-sensor and network data into

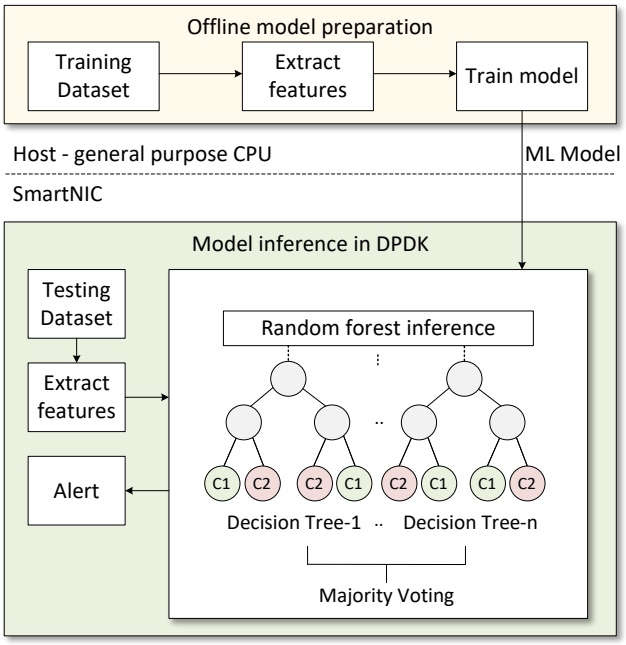


Fig. 2: Proposed system architecture. Features are extracted offline to train a RF model on the host, which is then offloaded to the SmartNIC for in-network inference.

normal and anomalous states, including both faults and cyber-attacks. Existing approaches range from distributed deep learning pipelines using auto-encoders and SVMs [9], to gradient-boosted decision trees for industrial robotics [10], and ML-based detection frameworks in CPS and ICS environments [11], [12]. While these methods achieve high detection accuracy, they typically rely on centralized execution on general-purpose CPUs or complex multi-stage inference pipelines, introducing non-negligible computational overhead and limiting their suitability for time-sensitive OT networks. Moreover, most prior work evaluates detection performance offline or at the application layer, without considering the cost of transporting high-rate industrial traffic and sensor data to the host for analysis. In contrast, our approach offloads RF inference directly to a SmartNIC using DPDK, enabling low-latency, deterministic anomaly detection within the network data plane and better aligning ML inference with the real-time requirements of CPS environments.

## IV. Proposed Framework

The proposed system follows a two-stage workflow that decouples offline model preparation from online in-network inference. As shown in Fig. 2, in the offline phase, raw CPS traces are processed on the host to extract flow-level statistical features of Modbus traffic. These features are used to train a RF classifier to distinguish normal operation from abnormal conditions. The trained model is then serialized into a compact JSON representation that encodes the structure and parameters of individual decision trees.

TABLE I: Dataset used for fault and cyber-attack detection

| Class | Network traffic (pkts) | | PLCs |
|---|---|---|---|
| | Modbus/TCP | Syslog | Sensor readings |
| **Normal** | 2863138 | 1128018 | 6534 |
| **Fault** | 1152195 | 535654 | 3806 |
| **Attack** | 19600574 | 59007 | N/A |
| **Total** | 23615907 | 1722679 | 10340 |

In the online phase, the model is offloaded to the SmartNIC and executed directly in the data plane using DPDK. RF inference is performed on the test data by evaluating each decision tree independently, and the final classification is obtained through majority voting across all trees. Based on the inferred class, the system triggers an alert for detected faults or attacks, enabling low-latency and host-independent detection within the network.

### A. Dataset

The proposed detection framework is evaluated using the publicly available dataset presented in [13]. The dataset was generated within an extended CPS environment and integrates three synchronized data sources: (i) physical process sensor readings from two PLCs, (ii) system logs collected from PLCs and HMIs, and (iii) Modbus/TCP network traffic captured in PCAP format. All data streams are timestamp-aligned, enabling correlation between process-level behavior and communication events.

Three categories of operational scenarios are considered: normal operation, system faults, and cyber-attacks. Normal operation corresponds to baseline process execution. System fault scenarios consist of five injected fault types, including sensor drift, tank leak, overheating caused by PLC delays, valve malfunction, and memory corruption. In addition, the dataset includes network-based attack scenarios such as Distributed Denial-of-Service (DDoS), Man-in-the-Middle (MitM), and port scanning attacks, captured in Modbus/TCP traffic.

Table I summarizes the composition of the dataset used in this work, including Modbus/TCP traffic, Syslog messages, and PLC sensor readings for normal, fault, and attack classes.

### B. Feature Engineering

Sensor measurements are parsed directly from Modbus packets and grouped into bidirectional communication flows. For each flow, only the first packets within a fixed window are considered to capture early behavioral characteristics to minimize feature extraction latency. From the extracted sensor values and packet timestamps, an initial set of eight statistical features is computed, including the minimum, maximum, mean, range, slope, and inter-arrival time statistics (*min_iat*, *max_iat*, and *mean_iat*). This baseline feature set is designed to capture both value distributions and short-term dynamics with minimal computational overhead.

To evaluate whether additional signal processing improves detection accuracy, a second feature set of eight features is

derived by transforming the sensor signals using Exponentially Weighted Moving Average (EWMA) and Cumulative Sum (CUSUM) techniques. EWMA provides a smoothed estimate of the signal trend and is computed as:

$$z_t = \alpha x_t + (1 - \alpha) z_{t-1}, \tag{1}$$

where $x_t$ is the sensor value at time $t$ and $\alpha \in (0, 1]$ controls the smoothing factor. CUSUM captures persistent deviations from a reference mean $\mu_0$ through cumulative positive and negative sums with a sensitivity parameter $k$:

$$C_t^+ = \max\big(0,\, C_{t-1}^+ + (x_t - \mu_0 - k)\big), \tag{2}$$

$$C_t^- = \min\big(0,\, C_{t-1}^- + (x_t - \mu_0 + k)\big), \tag{3}$$

### C. In-network RF Implementation in DPDK

After offline training, the selected RF model is deployed on the SmartNIC and executed entirely within the DPDK data plane. To enable efficient in-network inference, the trained model is serialized into a JSON representation that encodes the structure of each decision tree and leaf class labels. During initialization, the DPDK application safely parses this representation, enforces memory constraints, and constructs compact, heap-allocated node arrays for each tree. This setup allows subsequent packet processing to perform inference without dynamic memory allocation or host CPU involvement.

Each sample is evaluated independently by every decision tree in the forest. As shown in Algorithm 1, inference proceeds by traversing each tree from the root to a leaf node based on a sequence of feature-threshold comparisons. At each internal node, the feature value is compared against a stored threshold to determine whether the traversal follows the left or right child. Once a leaf node is reached, the corresponding class label is emitted as the tree's vote. The final prediction is obtained through majority voting across all trees, yielding a binary classification of normal or abnormal network behavior.

**Algorithm 1:** RF-Based Classification

**Input:** Feature vector $x$ (NUM_FEATURES),RF model
**Output:** Predicted fault/attack class
**Init:** Load trained RF model; each tree represented as a set of nodes
**foreach** *decision tree $T$ in the forest* **do**
  node ← root of $T$
  **while** *node is not a leaf* **do**
    $f$ ← node.feature
    $\theta$ ← node.threshold
    **if** $x[f] \leq \theta$ **then**
      node ← left_child(node)
    **else**
      node ← right_child(node)
  vote[node.class_label]++
predicted_class ← argmax(vote)
**return** predicted_class

TABLE II: Performance comparison across different window sizes for fault and attack detection.

| Task | Window Size | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|---|
| Fault Detection | 4 | 86.33 | 87.57 | 85.59 | 85.97 |
| | 8 | 87.47 | 88.17 | 86.93 | 87.23 |
| | 16 | 87.69 | 88.33 | 87.16 | 87.46 |
| | 32 | 87.91 | 88.39 | 87.46 | 87.71 |
| Attack Detection | 4 | 97.23 | 97.67 | 95.71 | 96.62 |
| | 8 | 97.96 | 98.51 | 96.64 | 97.52 |
| | 16 | 98.64 | 98.95 | 97.81 | 98.36 |
| | 32 | 98.57 | 98.93 | 97.68 | 98.28 |

## V. Implementation and Evaluation

This section evaluates the proposed SmartNIC-based anomaly detection for CPS, considering both fault and attack detection. A series of offline and online experiments were conducted to analyze the impact of feature dimensionality and model configuration on detection accuracy, inference latency, and memory footprint.

### A. Evaluating the Impact of Window Size

The impact of window size on classification performance is evaluated for both fault and attack detection using a set of 8 statistical features. Features are extracted from packets using window sizes of 4, 8, 16, and 32 packets. As shown in Table II, increasing the window size provides only marginal improvements across all evaluation metrics, including accuracy, precision, recall, and F1-score, for both tasks. Attack detection consistently achieves high performance across all window sizes, with all metrics exceeding 95%, while fault detection exhibits more modest gains. This difference is expected, as attack traffic introduces clearer and more distinguishable deviations in network behavior compared to faults.

In addition, both 8-feature and 16-feature configurations were evaluated, and only marginal improvements were observed with the extended feature set across all metrics. Given the limited performance gains and the strict latency constraints, the 8-feature configuration with a window size of 8 packets is selected for subsequent experiments.

### B. Evaluating Feature Importance and Explainability

To analyze the decision behavior of the RF models, feature importance is evaluated using SHAP (SHapley Additive exPlanations) for both fault and attack detection with the selected 8-feature configuration.

Fig. 3(a) and Fig. 3(b) show the SHAP summary plots for fault and attack detection, respectively. For fault detection, the extracted features correspond to Modbus sensor measurements and packet-level values, whereas for attack detection, features are derived from Modbus flow-level traffic statistics.

Inter-arrival time features (mean, minimum, and maximum) exhibit the highest impact on model predictions in both cases, indicating that timing characteristics are the primary indicators

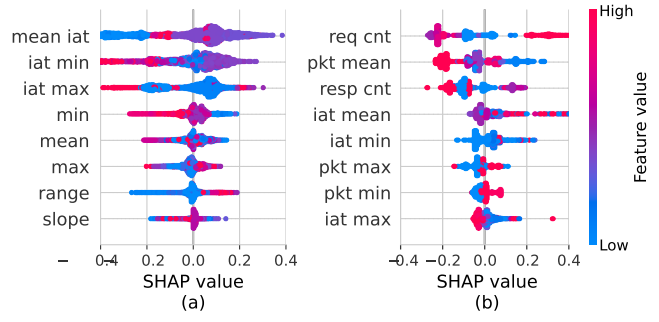


Fig. 3: SHAP summary plots for (a) fault detection using Modbus sensor/value features and (b) attack detection using flow-level features. The role of inter-arrival time is dominant across both tasks.

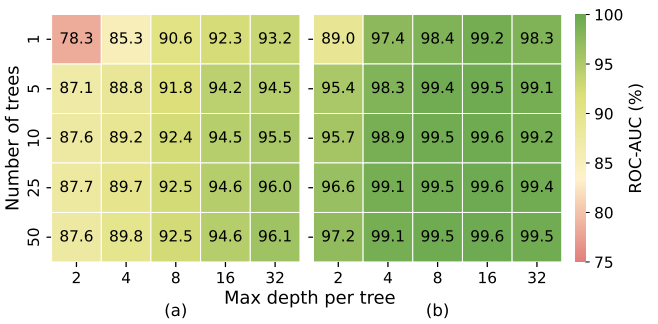


Fig. 4: ROC-AUC of RF models for (a) fault detection and (b) attack detection across varying numbers of trees and maximum tree depths.

of anomalous behavior. In contrast, packet- and value-based features contribute less significantly.

A clear distinction is observed between the two tasks. Attack detection shows more concentrated and consistent feature contributions, suggesting stronger separability between normal and anomalous traffic. In contrast, fault detection exhibits more dispersed feature effects, reflecting greater overlap between classes and increased classification complexity.

### C. Evaluating the Impact of the Model Size

Before offloading the RF models to the SmartNIC, the impact of key hyperparameters on detection performance is evaluated for both fault and attack detection. The number of trees (estimators) is varied from {1, 5, 10, 25, 50}, and the maximum depth per tree is varied from {2, 4, 8, 16, 32}. Model performance is assessed using the Receiver Operating Characteristic Area Under the Curve (ROC-AUC) metric.

Fig. 4 summarizes the ROC-AUC results across different model sizes for (a) fault detection and (b) attack detection. For both tasks, performance improves with increasing tree depth, as deeper trees capture more complex decision boundaries. However, distinct behaviors are observed.

Attack detection achieves high ROC-AUC values even with shallow trees and a small number of estimators, indicating strong separability between normal and anomalous traffic. In contrast, fault detection requires deeper trees to reach comparable performance, reflecting the increased complexity and overlap between normal and faulty behavior.

Increasing the number of trees provides additional performance gains for both tasks; Deeper trees are also associated with increased inference latency, as more decision nodes must be traversed during classification. For fault detection, a maximum tree depth of 32 is selected to capture the higher complexity of the data. In contrast, shallower trees are sufficient for attack detection, and a maximum depth of 8 is selected. The selected models are subsequently evaluated on the SmartNIC to analyze the trade-offs between detection accuracy, inference latency, and memory footprint in the data plane.

### D. Evaluating the Inference Time in the Data Plane

To evaluate runtime performance, the selected RF models are offloaded to the SmartNIC and executed entirely in the data plane. For fault detection, models with a maximum tree depth of 32 and varying numbers of trees ({1, 5, 10, 25, 50}) are considered. For attack detection, shallower models with a maximum depth of 8 are deployed.

Fig. 5(a) and Fig. 5(b) show the cumulative distribution functions (CDFs) of inference latency for fault and attack detection, respectively. In both cases, increasing the number of trees leads to higher inference latency, as more trees must be traversed during classification.

The latency distribution also widens with larger ensembles. This variability arises from the structure of RF, where individual trees may terminate early depending on feature values. Samples that reach leaf nodes closer to the root require fewer comparisons, resulting in shorter inference paths, while deeper traversals across multiple trees incur higher latency.

A clear distinction is observed between the two tasks. Fault detection exhibits higher latency and greater variability due to deeper trees, whereas attack detection achieves consistently low latency with tighter distributions, owing to the use of shallow trees.

These results highlight the impact of model structure on inference efficiency and demonstrate that lightweight models enable microsecond-scale processing in the SmartNIC data plane.

### E. Evaluating Online Accuracy and Memory Footprint

The impact of model size on online classification accuracy and memory footprint is evaluated for both fault and attack detection. Table III summarizes the results and compares online performance with offline training accuracy.

For both tasks, increasing the number of trees improves online accuracy due to a stronger ensemble effect. However, the magnitude of improvement differs. Fault detection shows noticeable gains as the ensemble size increases, reflecting the higher complexity of the task. In contrast, attack detection achieves consistently high accuracy even with a small number of trees, with only marginal improvements observed for larger models.

TABLE III: Online and offline performance comparison for RF models for fault and attack detection.

| Task | Trees | Inference Time ($\mu$s) | | Accuracy (%) | | Memory Footprint (MiB) |
|---|---|---|---|---|---|---|
| | | Mean | Std. Dev. | Offline | Online | |
| Fault Detection | 1 | 0.3 | 2.3 | 85.86 | 78.13 | 0.78 |
| | 5 | 3.4 | 5.0 | 88.17 | 83.76 | 3.50 |
| | 10 | 8.8 | 8.8 | 88.75 | 85.20 | 7.19 |
| | 25 | 27.7 | 22.3 | 89.22 | 87.43 | 18.08 |
| | 50 | 66.0 | 47.3 | 89.25 | 88.36 | 36.15 |
| Attack Detection | 1 | 0.11 | 2.34 | 98.71 | 98.25 | 0.0043 |
| | 5 | 0.20 | 2.26 | 98.85 | 98.46 | 0.0226 |
| | 10 | 0.33 | 2.54 | 98.93 | 98.55 | 0.0519 |
| | 25 | 0.79 | 2.83 | 98.95 | 97.89 | 0.1506 |
| | 50 | 1.61 | 3.37 | 98.97 | 97.91 | 0.2953 |

Memory footprint increases linearly with the number of trees, as each tree must be stored in SmartNIC memory. This overhead is more pronounced for fault detection due to the use of deeper trees than what attack detection requires.

A gap between offline and online accuracy is observed, particularly for fault detection, where online performance is lower. This difference highlights the importance of evaluating models under realistic in-network conditions, where timing variability and partial flow visibility affect classification outcomes.

## VI. Conclusion and Future Work

This paper presented a SmartNIC-accelerated anomaly detection framework for CPS that offloads Random Forest (RF) inference directly into the network data plane using DPDK, enabling real-time detection of faults and network-based attacks. A systematic evaluation of window size, feature selection, and model configuration identified a lightweight feature set and efficient model architecture that balance detection accuracy with ultra-low-latency execution.

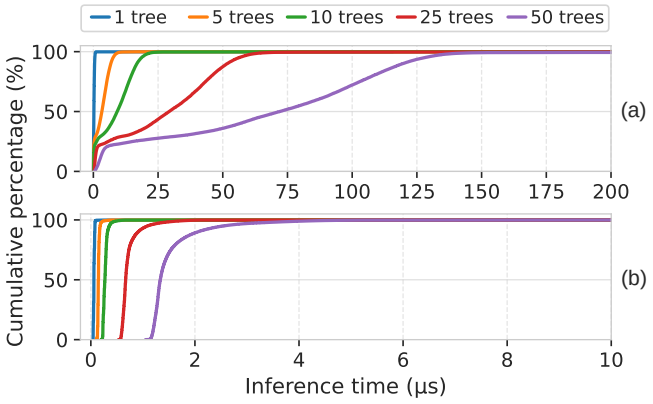


Fig. 5: CDF of inference latency for RF models with varying numbers of trees executed in the SmartNIC data plane for (a) fault detection and (b) attack detection.

Experimental results on an NVIDIA BlueField-3 SmartNIC show that in-network inference achieves high accuracy with microsecond-scale latency and low memory footprint. Attack detection exceeds 98% accuracy with shallow trees (depth 8), sub-microsecond latency (0.1–1.6 $\mu$s), and less than 0.3 MiB memory, while fault detection reaches up to 88% online accuracy using deeper trees (depth 32), with latency up to 66 $\mu$s and memory usage up to 36 MiB.

As future work, adaptive and incremental learning will be explored to handle evolving system behavior without full retraining. Additionally, hardware-assisted acceleration of tree traversal and feature extraction will be investigated, along with support for additional industrial protocols and more complex CPS deployments.

## Acknowledgment

The work was supported by the National Science Foundation, under awards 2403360 and 2400729. The authors would also like to acknowledge the FABRIC [14] team.

## References

[1] K. Stouffer, K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule *et al.*, "Guide to operational technology (OT) security," 2023.

[2] S. Elizalde, A. AlSabeh, A. Mazloum, S. Choueiri, E. Kfoury, J. Gomez, and J. Crichigno, "A survey on security applications with smartnics: Taxonomy, implementations, challenges, and future trends," *Journal of Network and Computer Applications*, p. 104257, 2025.

[3] samiachoueiri Github, "DPDK-ICS-RF," [Online]. Available: https://github.com/samiachoueiri/DPDK-ICS-RF, 2026.

[4] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & information systems engineering*, 2014.

[5] E. F. Kfoury, S. Choueiri, A. Mazloum, A. AlSabeh, J. Gomez, and J. Crichigno, "A comprehensive survey on SmartNICs: architectures, development models, applications, and research directions," *IEEE Access*, 2024.

[6] S. Choueiri, A. Mazloum, S. Elizalde, E. F. Kfoury, and J. Crichigno, "Volumetric DDoS Attack Detection and Mitigation using P4-DPDK," in *2025 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, 2025, pp. 1–6.

[7] R. Samson Raj and D. Jin, "Leveraging data plane programmability towards a policy-driven in-network security framework for industrial control systems," in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems*, 2025, pp. 152–163.

[8] F. E. R. Cesen and C. E. Rothenberg, "Offloading Robotic and UAV applications to the network using programmable data planes," in *2023 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE, 2023, pp. 207–212.

[9] S. U. Jan, Y. D. Lee, and I. S. Koo, "A distributed sensor-fault detection and diagnosis framework using machine learning," *Information Sciences*, 2021.

[10] P. Doshi, N. Daftary, H. Jani, and A. Nair, "CatBoost-driven anomaly detection in industrial robotic arms Using CASPER dataset," *IEEE Sensors Letters*, 2026.

[11] S. Elizalde, S. Choueiri, A. Mazloum, J. Gomez, E. Kfoury, and J. Crichigno, "Accelerating anomaly detection in industrial control systems using SmartNICs and DPDK," in *IEEE Consumer Communications & Networking Conference (CCNC)*, Las Vegas, NV, USA, Jan. 2026.

[12] A. F. Amiri, H. Oudira, A. Chouder, and S. Kichou, "Faults detection and diagnosis of PV systems based on machine learning approach using random forest classifier," *Energy Conversion and Management*, 2024.

[13] D. Sudyana, "ICS Dataset," Sep. 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17165850

[14] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, K.-C. Wang, T. Lehman, and P. Ruth, "Fabric: A national-scale programmable experimental network infrastructure," *IEEE Internet Computing*, vol. 23, no. 6, pp. 38–47, 2020.